Overlooked Area Detector: Technical Blueprint

Nov 7, 2026 · @shriii

A web app that finds the neighborhoods a disaster hit hardest but that are not asking for help, using only public hazard, claims and census data, so State Farm can send catastrophe teams there first. The demo replays Hurricane Harvey (August 2017) in Harris County, Texas.

1. Application framework

The app is a four-layer system: an offline Python data pipeline builds one database file, a FastAPI backend serves it, a React map dashboard shows it, and a small set of models scores every ZIP code each day of a disaster. Nothing depends on victims replying, uploading photos, or private State Farm data.

The core idea in one line

For every ZIP code and every day after landfall, compare **how many households should be asking for help** (predicted from hazard, housing and vulnerability) with **how many actually are** (FEMA assistance registrations by date). ZIPs that are hit hard but unusually quiet are the overlooked ones.

Components

Layer	What it is	What it does
Data pipeline	Python scripts (pandas, GeoPandas, rasterstats) run once before the demo	Downloads public data, joins it to ZIP codes, writes `oad.duckdb`
Database	DuckDB file (embedded, no server)	Stores ZIP shapes, hazard, vulnerability, daily registrations, predictions, scores
ML layer	scikit-learn models in `ml/`	Predicts expected need per ZIP; computes daily gap scores

Decision engine	Plain Python rules + statistics	Turns scores into a ranked priority list, action types and team locations
Backend API	FastAPI	Serves GeoJSON, rankings, time series, plans, briefs over REST
Frontend	React + TypeScript + MapLibre	Map, day slider, priority queue, ZIP detail, plan and backtest pages
External live feeds (optional)	NWS alerts API, NHC GIS	"Live watch" mode for a current storm

[embed: node/8a104108-5335]

Public sources are downloaded once and turned into one database file; at demo time the API only reads that file, and live feeds are an optional extra.

How the system operates

1. **Build time (before the hackathon demo):** the pipeline pulls FEMA, NOAA, USGS and CDC data for Hurricane Harvey, aggregates everything to ZIP Code Tabulation Areas (ZCTAs), trains the expected-need model, and precomputes a score for every ZIP for days 1 to 30 after landfall.
2. **Run time (during the demo):** the user picks the event and drags the day slider. The frontend asks the API for that day; the API reads precomputed rows from DuckDB and returns them in under 100 ms. The only things computed live are the team-placement plan (a fast greedy algorithm) and the optional briefing text.
3. **Why precompute:** hackathon Wi-Fi fails, public APIs throttle, and judges click fast. Precomputing makes the demo instant and lets the whole app run offline from one file.

Real, historical, simulated and predicted data

Every number in the app carries one of four labels, shown as a small badge in the UI:

- **Real historical:** FEMA registrations, NFIP claims, USGS high-water marks, NOAA rainfall, CDC SVI for Harvey
- **Real live:** NWS alerts and NHC storm data, only in Live watch mode
- **Predicted:** expected need, gap scores, claim-surge forecasts
- **Simulated:** candidate team sites if OpenStreetMap is unavailable, and any State Farm policyholder counts used to illustrate the business case

2. Feature breakdown

Ten features make up the app. F1 to F6 are the MVP; F7 to F9 make the demo stand out; F10 is a stretch. They form one chain: hazard and vulnerability feed expected need, expected need plus observed registrations give the gap, the gap drives priorities, priorities drive team placement and the briefing.

#	Feature	Tier	Difficulty
F1	Event replay and day slider	MVP	Easy
F2	Hazard impact layer	MVP	Medium
F3	Expected-need model	MVP	Medium
F4	Response-gap detector	MVP (core)	Medium
F5	Vulnerability lens and "why flagged"	MVP	Easy
F6	Priority queue with recommended actions	MVP	Easy
F7	Catastrophe team placement optimizer	Impressive	Medium
F8	Backtest: "what we would have caught"	Impressive	Medium
F9	Claim-surge forecast and deployment brief	Impressive	Easy to medium
F10	Live watch mode	Stretch	Medium

F1. Event replay and day slider

- **What and why:** Lets the user pick a past disaster and move day by day from landfall. Judges see how the picture changes over time, which is the whole point: overlooked areas are visible early, before help arrives.
- **Interaction:** Event dropdown in the top bar; a slider from Day 0 to Day 30 with a play button that advances one day per second.
- **Internals:** Changing the day sets `day` in the React state; TanStack Query fetches `/events/{id}/map?day=N` and `/events/{id}/priorities?day=N`, caching each day so scrubbing back is instant.
- **Data:** `events` table (name, FEMA disaster number, landfall date, bounding box); every other table keyed by `event_id` and `day`.
- **Algorithms/APIs:** None beyond lookups.
- **Backend:** Reads precomputed rows; no computation.
- **Frontend:** Slider, date label ("Day 4 · Aug 29, 2017"), play/pause, KPI strip that updates per day.
- **Output:** The day context every other feature uses.
- **Links:** Drives F2 to F9.
- **Difficulty:** Easy.

F2. Hazard impact layer

- **What and why:** Shows how hard each ZIP was physically hit. Without it, a quiet ZIP could simply be undamaged.
- **Interaction:** Map layer toggle "Hazard"; hovering a ZIP shows rainfall, nearby high-water marks and modeled damaged buildings.
- **Internals (pipeline, step by step):**
    1. Download NOAA Stage IV daily rainfall GeoTIFFs for Aug 25 to Sep 1, 2017 and sum them into one storm-total raster.
    2. Compute the mean and max rainfall inside each ZIP polygon with `rasterstats.zonal_stats`.
    3. Load USGS high-water marks (points with height above ground); count marks and take the max height within each ZIP.
    4. Load FEMA's modeled damage assessment points (published Aug 27 to Sep 2, 2017); count points per ZIP per day, divided by housing units.
    5. Combine into a 0 to 1 `hazard_index` = mean of percentile ranks of the three measures.
- **Data:** Stage IV rainfall, USGS STN high-water marks, FEMA Harvey damage assessments, Census ZCTA polygons.
- **Algorithms:** Zonal statistics, point-in-polygon spatial join, percentile ranking.
- **Backend:** Serves `zip_hazard` rows joined to geometry.
- **Frontend:** Blue sequential choropleth; legend in inches of rain.
- **Output:** `hazard_index` and raw measures per ZIP.
- **Links:** Main input to F3; shown beside the gap in F5.
- **Difficulty:** Medium (raster handling is the hardest data step).

F3. Expected-need model

- **What and why:** Predicts how many households in each ZIP will end up needing assistance. This is the baseline the gap is measured against.
- **Interaction:** Not clicked directly; appears as "Expected households needing help" in ZIP details and the KPI strip.
- **Internals:** A Poisson regression (with a gradient-boosted alternative) predicts final FEMA registrations reporting home damage per ZIP from hazard, housing and vulnerability features. Predictions are made out-of-fold so a ZIP never predicts itself (Section 8).
- **Data:** Features from F2 and F5, housing units from CDC SVI, historical NFIP flood claims per ZIP before 2017; target from FEMA IHP registrations.
- **Algorithms:** scikit-learn `PoissonRegressor` with log(housing units) as exposure; `HistGradientBoostingRegressor(loss="poisson")` as challenger; `GroupKFold` spatial cross-validation.
- **Backend:** Pipeline writes `predictions`; API reads them.
- **Frontend:** Number plus an uncertainty range in ZIP detail.
- **Output:** `expected_final` per ZIP.
- **Links:** Feeds F4 and F9.
- **Difficulty:** Medium.

F4. Response-gap detector (the core)

- **What and why:** Flags ZIPs where far fewer households have asked for help by day N than expected. This answers "make sure no one is overlooked."
- **Interaction:** Default map layer "Gap"; red ZIPs are quiet relative to expectation. A badge marks statistically significant gaps.
- **Internals:**
    1. Count registrations per ZIP per day from `appliedDate`, cumulative to day N.
    2. Compute the event-wide pace R_N = total observed by day N ÷ total expected final. This adjusts for the whole region registering slowly at first.
    3. Expected by day N for a ZIP = expected_final × R_N.
    4. Gap ratio = observed ÷ expected_by_day; gap score = 1 − gap ratio, floored at 0.
    5. Poisson test: p = P(X ≤ observed | λ = expected_by_day). Significant when p < 0.01 and expected_by_day ≥ 20.
- **Data:** `registrations_daily`, `predictions`.
- **Algorithms:** Ratio estimator, Poisson lower-tail test (`scipy.stats.poisson.cdf`).
- **Backend:** Precomputed into `gap_scores` for days 1 to 30.
- **Frontend:** Red sequential choropleth, significance outline, tooltip "Expected 1,180 · Observed 410 · 65% quiet."
- **Output:** `gap_score`, `unmet_estimate` = expected_by_day − observed, `p_value`.
- **Links:** Main input to F6, F7, F8.
- **Difficulty:** Medium (easy math, careful data prep).

F5. Vulnerability lens and "why flagged"

- **What and why:** Explains who lives in a flagged ZIP and why it was flagged, so a responder knows what kind of help to send.
- **Interaction:** Clicking a ZIP opens a side panel: SVI bars (65+, no vehicle, limited English, mobile homes, poverty), a contributions chart for the prediction, and the registration curve versus expected.
- **Internals:** Tract-level SVI is aggregated to ZIPs, weighted by housing units in each tract-ZIP overlap. "Why" shows each feature's contribution to the log prediction (coefficient × standardized value) for the Poisson model.
- **Data:** CDC/ATSDR SVI 2016 (pre-Harvey), Census ZCTA-to-tract relationship file.
- **Algorithms:** Weighted aggregation; linear contribution decomposition.
- **Backend:** `GET /events/{id}/zips/{zip}?day=N` returns profile, contributions and daily series.
- **Frontend:** Panel with bars, a line chart and plain-language reasons.
- **Output:** Reasons list, e.g. "28% of households have no vehicle (state 92nd percentile)."
- **Links:** Supplies the vulnerability multiplier and action rules for F6.
- **Difficulty:** Easy.

F6. Priority queue with recommended actions

- **What and why:** Turns scores into a ranked to-do list for the catastrophe team: which ZIPs first, and what to do there.
- **Interaction:** Right-hand list, top 20 ZIPs; each card shows rank, ZIP, area name, unmet estimate, action chips; clicking flies the map there.
- **Internals:** Priority = unmet_estimate × (1 + 0.5 × SVI) × significance weight (1.0 if significant, else 0.5). Action rules map vulnerability patterns to response types (Section 9).
- **Data:** `gap_scores`, ZIP vulnerability profile.
- **Algorithms:** Weighted scoring and a rule table.
- **Backend:** `GET /events/{id}/priorities?day=N&limit=20`.
- **Frontend:** Ranked cards; filter chips ("significant only," "65+ heavy").
- **Output:** Ranked list with actions; exportable to CSV.
- **Links:** Input to F7 and F9.
- **Difficulty:** Easy.

F7. Catastrophe team placement optimizer

- **What and why:** State Farm deploys mobile claims units and catastrophe teams. This picks the best k sites so the most priority-weighted households are within reach, avoiding flooded sites.
- **Interaction:** "Plan deployment" page: choose k (1 to 10) and radius (5 to 15 km), click Run; sites appear as pins with coverage rings.
- **Internals:**
    1. Candidate sites: large parking lots, schools and places of worship from OpenStreetMap (Overpass API), pre-downloaded.
    2. Drop candidates in ZIPs with hazard_index > 0.8 or within 300 m of a high-water mark above 2 ft.

3. Weighted maximum coverage by greedy selection: repeatedly add the site covering the most still-uncovered priority weight within the radius.

- **Data:** `sites`, `gap_scores`, ZIP centroids.
- **Algorithms:** Greedy max-coverage (guaranteed within about 63% of optimal), haversine distance; road travel time via OSMnx as a stretch.
- **Backend:** `POST /events/{id}/plan` computes live in under a second.
- **Frontend:** Pins, rings, a table of sites with households covered.
- **Output:** Site list and percentage of priority weight covered.
- **Links:** Uses F6; feeds F9.
- **Difficulty:** Medium.

F8. Backtest: "what we would have caught"

- **What and why:** Proves the method works using what happened later. This is the most persuasive slide for judges.
- **Interaction:** "Validation" page: precision@10 and @25 for the gap ranking versus baselines, plus a map of correctly flagged ZIPs.
- **Internals:** Define overlooked ZIPs independently of the model: top 30% damage density (NFIP paid claims + modeled damage per housing unit) and bottom third of the share of final registrations received by day 5. Compare rankings at day 3, 5 and 7 against baselines: rainfall only, SVI only, raw registrations.
- **Data:** NFIP claims for Harvey, FEMA damage assessments, registrations.
- **Algorithms:** Precision@k, Spearman correlation.
- **Backend:** Precomputed `backtest` table; `GET /events/{id}/backtest`.
- **Frontend:** Metric tiles, small bar chart, map.
- **Output:** "At day 5, 7 of our top 10 were truly overlooked; rainfall alone found 3." (Illustrative; your numbers come from the run.)
- **Links:** Evaluates F3 and F4.
- **Difficulty:** Medium.

F9. Claim-surge forecast and deployment brief

- **What and why:** Tells managers how many claims and visits to staff for, and packages the day's plan into a one-page brief.
- **Interaction:** "Brief" button exports a PDF or Markdown page; a chart shows forecast cumulative need versus observed.
- **Internals:** Remaining need per ZIP = expected_final − observed; summed for the region; staffing = remaining ÷ visits per adjuster per day (a user setting, default 6). The brief is filled from a template; optionally Claude rewrites the summary paragraph from the same JSON.
- **Data:** `predictions`, `gap_scores`, plan output.
- **Algorithms:** Arithmetic; optional LLM summarization with a fixed prompt.
- **Backend:** `GET /events/{id}/brief?day=N&format=pdf`.
- **Frontend:** Forecast chart and download button.
- **Output:** A brief listing top ZIPs, actions, sites and staffing.
- **Links:** Consumes F3, F6, F7.
- **Difficulty:** Easy to medium.

F10. Live watch mode (stretch)

- **What and why:** Shows the same pipeline pointed at a current storm: active NWS warnings and the NHC cone over Texas, with ZIPs ranked by expected need before anyone registers.
- **Interaction:** "Live" tab.
- **Internals:** Poll `api.weather.gov/alerts/active?area=TX`; intersect alert polygons with ZIPs; run F3 using forecast rainfall in place of observed. The gap part cannot run live on public data because FEMA publishes registrations only for disasters declared more than 30 days ago; in production State Farm would plug in its own first-notice-of-loss stream.
- **Data:** NWS alerts, NHC GIS shapefiles, Stage IV hourly rainfall.
- **Difficulty:** Medium; skip unless MVP is done.

3. Application workflow

The app runs in two phases: a build phase that turns raw public data into scored ZIPs once, and a demo phase where every click is a fast database read. The diagram under section 17 shows the full data flow; the steps below trace each user action.

Build phase (run once, about 1 to 2 hours of compute on a laptop)

1. `pipeline/01_download.py` pulls each source into `data/raw/` (FEMA by API, others by file download).
2. `02_geography.py` loads Harris County area ZCTAs and the ZCTA-to-tract relationship file.
3. `03_features.py` builds `zip_static` (SVI, housing units, historical claims, NRI) and `zip_hazard` (rainfall, high-water marks, damage points).
4. `04_registrations.py` turns raw FEMA registrations into daily counts per ZIP.
5. `ml/train_expected.py` trains the model with spatial cross-validation and writes out-of-fold `predictions`.
6. `06_scores.py` computes `gap_scores` and priorities for days 1 to 30.
7. `07_backtest.py` computes validation metrics.
8. `08_export.py` writes `oad.duckdb` and static JSON snapshots for offline mode.

Demo phase, action by action

User action	Frontend	Backend	Database	Result
Opens the app	Loads React bundle; calls `GET /events`	Lists events	`SELECT * FROM events`	Landing page with event cards

Selects "Hurricane Harvey"	Routes to `/event/harvey-2017`; fetches event, map day 3, priorities day 3	Joins geometry with day-3 scores	Reads `zips`, `gap_scores`	Map zooms to Houston; red ZIPs appear; KPI strip fills
Drags slider to day 5	Debounced fetch for day 5 (cached after first time)	Same queries, `day = 5`	Indexed read on `(event_id, day)`	Map recolors; queue reorders
Switches layer to Hazard or Vulnerability	Swaps the fill property; no new request (all metrics come in one GeoJSON)	None	None	Map recolors instantly
Clicks a ZIP	Opens side panel; fetches detail	Builds profile, contributions, daily series	Reads `zip_static`, `zip_hazard`, `registrations_daily`, `predictions`	Panel with reasons, curve and actions
Clicks "Plan deployment," k = 4, radius 10 km	`POST /plan`	Runs greedy max-coverage on that day's priorities and eligible sites	Reads `gap_scores`, `sites`; writes `plans`	Pins, rings, coverage %
Opens Validation	Fetches backtest	Returns metrics	Reads `backtest`	Precision tiles and comparison chart
Clicks "Export brief"	Downloads file	Fills template; optional LLM	Reads plan and priorities	One-page brief

summary;
renders PDF

What "triggers an analysis" means here

The scoring is already done for every day, so the only analysis a user triggers live is the deployment plan (F7) and the brief (F9). This is deliberate: a judge sees results instantly, and the heavy steps were tested before the demo.

4. Frontend and UI/UX

Use React with TypeScript, built with Vite, MapLibre GL for the map, Recharts for charts, and Tailwind CSS with shadcn/ui components. This gives a polished, fast map dashboard with no paid map tokens.

Why this stack, and the alternative

Option	Pros	Cons	Verdict
React + MapLibre + FastAPI	Polished, smooth map, full control, impressive demo	Two codebases; needs one person comfortable with React	**Recommended** if the team has 3+ people
Streamlit + Folium	One Python file, very fast to build	Reruns the whole page on each click; looks generic; slider feels laggy	Fallback if the team is 1 to 2 people

MapLibre GL is the free, open-source fork of Mapbox GL: it renders thousands of ZIP polygons on the GPU and needs no API key. Use free CARTO "Positron" or "Dark Matter" basemap tiles, or OpenFreeMap. Fetching is handled by TanStack Query (caching, loading states).

Pages and navigation

A left sidebar with five items: Overview, Map, Plan, Validation, Brief. The event and day live in the URL (`/event/harvey-2017/map?day=5`), so a judge can bookmark a view and the back button works.

Page 1: Landing / event picker

- **Layout:** Hero line ("Find the neighborhoods the storm hit hardest that aren't asking for help"), then event cards.
- **Elements:** Card per event: name, dates, counties, a thumbnail of its day-5 gap map, "Open" button. A data-sources footer with badges.

Page 2: Map dashboard (the main screen)

- **Top bar:** Event name, day slider (Day 0 to 30) with play/pause and date label, layer switch (Gap · Hazard · Vulnerability · Expected need), "Significant only" toggle.
- **KPI strip (4 tiles):** Households expected to need help · Registered so far · Estimated unmet · ZIPs significantly quiet. Each tile shows its data badge (predicted or real).
- **Map (center, about 65% width):** ZIP choropleth for the chosen layer; thick outline on significant gaps; high-water marks as small dots (toggle); deployment pins when a plan exists. Hover tooltip: ZIP, area, expected vs observed, gap %. Click opens the side panel.
- **Priority queue (right, about 35%):** Top 20 cards: rank, ZIP and area name, unmet estimate, gap %, action chips ("Welfare checks · 65+", "Spanish outreach", "Mobile-home inspections"). Click flies the map to the ZIP. "Export CSV" button.
- **ZIP detail panel (slides over the queue):** Header with ZIP and gap badge; a line chart of observed cumulative registrations vs expected curve with a marker at the current day; "Why flagged" list; vulnerability bars vs state median; hazard numbers; recommended actions.

Page 3: Plan deployment

- **Controls:** Number of teams k (slider 1 to 10), coverage radius (5 to 15 km), "Exclude flooded sites" toggle, Run button.
- **Output:** Map with site pins and rings; table of sites (name, ZIP, households covered, top ZIPs served); a coverage gauge ("68% of priority need within 10 km").

Page 4: Validation

- **Tiles:** Precision@10 and @25 at day 3, 5, 7.
- **Chart:** Grouped bars, our method vs rainfall only vs SVI only vs raw registrations.
- **Map:** True positives green, misses grey, false alarms orange, with a one-line explanation of the definition of "overlooked."

Page 5: Brief

- A preview of the one-page brief and "Download PDF" and "Copy summary" buttons; staffing input (visits per adjuster per day).

Making it look professional

- **One accent color for the core signal:** the gap layer in a red sequential scale; everything else neutral grey and blue. Judges should see the overlooked ZIPs before reading anything.
- **Dark map theme for the demo screen:** red gap areas read clearly on a projector.

- **Animated playback:** the play button that animates days 0 to 10 is the most memorable 15 seconds of the demo.
- **Data badges everywhere:** small "Real · FEMA" or "Predicted" tags make the app look trustworthy and answer the judges' first question.
- **Number formatting:** thousands separators, "~" on predictions, no more than 2 significant decimals.
- **Loading skeletons** instead of spinners; an "Offline snapshot" badge when running from static JSON.
- **Accessibility:** colorblind-safe palette (red sequential plus outline for significance, not color alone).

5. Backend

Use FastAPI (Python). The data science is already in Python, so the API can import the same scoring code; FastAPI gives typed request/response models with Pydantic and free interactive docs at `/docs`, which doubles as a demo of the API for judges. Flask is simpler but has no built-in validation or docs; Node would split the team across two languages.

Modules

Module	Responsibility
`api/main.py`	App setup, CORS, routers, startup loads the DuckDB connection read-only
`api/routers/events.py`	Event list and metadata
`api/routers/maps.py`	GeoJSON for a day with all layer metrics
`api/routers/zips.py`	ZIP detail: profile, contributions, time series
`api/routers/plan.py`	Deployment optimizer endpoint
`api/routers/validation.py`	Backtest metrics

`api/routers/brief.py`	Brief generation (Markdown, PDF)
`api/routers/live.py`	Optional live NWS alerts proxy with 5-minute cache
`core/db.py`	DuckDB connection and query helpers
`core/scoring.py`	Gap, priority and action rules (shared with the pipeline)
`core/optimizer.py`	Greedy max-coverage
`core/brief.py`	Jinja2 template, optional LLM call, WeasyPrint PDF
`core/schemas.py`	Pydantic response models

Endpoints

Method	Path	Returns	Feature
GET	`/health`	`{"status":"ok","mode":"db"}`	Ops
GET	`/events`	List of events	F1
GET	`/events/{event_id}`	Event metadata, day range, bbox	F1

GET	`/events/{event_id}/map?day=5`	GeoJSON FeatureCollection, one feature per ZIP with all layer metrics	F2, F4, F5
GET	`/events/{event_id}/kpis?day=5`	Expected, registered, unmet, significant count	F1, F4
GET	`/events/{event_id}/priorities?day=5&limit=20&significant_only=true`	Ranked ZIPs with actions	F6
GET	`/events/{event_id}/zips/{zip}?day=5`	Profile, reasons, daily series	F3, F5
POST	`/events/{event_id}/plan`	Selected sites and coverage	F7
GET	`/events/{event_id}/backtest`	Precision@k by day and method	F8
GET	`/events/{event_id}/forecast?day=5`	Regional observed vs expected curve, staffing	F9
GET	`/events/{event_id}/brief?day=5&format=pdf`	File download	F9
GET	`/live/alerts?state=TX`	Active NWS alerts intersected with ZIPs	F10

Sample responses

```
GET /events/harvey-2017/priorities?day=5&limit=2
```

JSON

```
{
  "event_id": "harvey-2017",
  "day": 5,
  "date": "2017-08-30",
  "items": [
    {
      "rank": 1,
      "zip": "77xxx",
      "area": "Example neighborhood",
      "expected_by_day": 1178,
      "observed": 410,
      "gap_score": 0.65,
      "unmet_estimate": 768,
      "p_value": 1.2e-120,
      "significant": true,
      "svi": 0.82,
      "priority": 1083,
      "actions": ["door_to_door_canvass",
"welfare_check_65plus"],
      "data_labels": {"observed": "real", "expected_by_day":
"predicted"}
    }
  ]
}
```

(Values illustrative until the pipeline runs.)

`POST /events/harvey-2017/plan` request and response:

JSON

```
{"day": 5, "k": 4, "radius_km": 10, "exclude_flooded": true}
```

JSON

```
{
  "plan_id": "p_20171230_k4",
  "sites": [
    {"site_id": "osm_123", "name": "Example school lot", "lat":
29.80, "lon": -95.30,
```

```
    "covered_zips": ["77xxx", "77yyy"], "covered_priority": 2140}
  ],
  "coverage_pct": 0.68
}
```

How the backend serves each feature

- **Reads (F1 to F6, F8):** single SQL queries against indexed tables; GeoJSON geometry is pre-simplified (0.0005 degree tolerance) at build time so a full-region response stays under 2 MB.
- **Compute (F7):** loads the day's priorities and eligible sites into NumPy arrays, builds a site-by-ZIP distance matrix (haversine), runs greedy selection; about 50 ms for 500 sites × 600 ZIPs.
- **Generation (F9):** renders a Jinja2 template; if `ANTHROPIC_API_KEY` is set, asks Claude for a 120-word summary from the JSON with instructions to use only the given numbers; otherwise uses the template sentence.
- **Caching:** `functools.lru_cache` on `(event_id, day)` for map and priorities.
- **Offline fallback:** if `OFFLINE=1`, routers serve the JSON snapshots from `data/snapshots/` instead of DuckDB.

6. APIs, public datasets and external sources

Every source below was checked against its documentation on Nov 7, 2026 for the fields this app needs. All are free. Only two optional ones take a key, and none need one for hackathon volumes.

The most important discovery: FEMA's **IHP Valid Registrations** dataset records the **date each household applied** (`appliedDate`), its ZIP, census block group, age band, residence type, registration method, water level and FEMA-verified damage. That one public dataset makes the gap detector possible without any private data.

Summary

Source	Gives	Feature	Cost / key	Type
OpenFEMA IHP Valid Registrations v2	Applicant-level registrations with date, ZIP, damage	F3 target, F4, F8	Free, no key	Historical

OpenFEMA NFIP Redacted Claims v2	Flood insurance claims by ZIP, tract, event	F3 feature, F8 ground truth	Free, no key	Historical
OpenFEMA API documentation	Paging, filters, formats	All FEMA pulls	Free, no key	—
FEMA Harvey Damage Assessments (HydroShare)	Modeled damaged-building points by day, Aug 27 to Sep 2	F2, F8	Free	Historical (real-time during event)
NOAA/NWS Stage IV precipitation	Daily rainfall GeoTIFFs	F2	Free, no key	Historical and live
USGS STN Flood Event Data	High-water marks with height above ground	F2, F7	Free, no key	Historical
CDC/ATSDR SVI	Tract vulnerability (65+, no vehicle, limited English, mobile homes, housing units)	F3, F5, F6	Free	Historical
Census ZCTA relationship files	ZIP-to-tract overlaps	Geography join	Free	Static
FEMA National Risk Index	Tract flood risk, expected annual loss	F3	Free	Static
NHC GIS data	Storm track, cone, wind swath	F10, map context	Free	Live and archive

NWS API	Active alerts with polygons	F10	Free, User-Agent header	Live
Census Data API (ACS)	Extra demographics if needed	F5 optional	Free; key optional	Historical
USGS Water Data APIs	Stream gauges	F10 optional	Free; key optional	Live and historical
OpenStreetMap via Overpass / OSMnx	Candidate sites (lots, schools, worship)	F7	Free, fair use	Static

6.1 OpenFEMA IHP Valid Registrations v2 (the core source)

- **Endpoint:** `https://www.fema.gov/api/open/v2/IndividualsAndHouseholdsProgramValidRegistrations`
- **Provides:** One row per valid applicant. Fields used: `disasterNumber`, `declarationDate`, `appliedDate`, `damagedZipCode`, `censusGeoid` (block group), `applicantAge` (bands incl. 65+), `householdComposition`, `ownRent`, `residenceType` (incl. M = mobile home), `registrationMethod` (call center, internet, mobile), `homeDamage`, `waterLevel` (inches), `destroyed`, `rpfvl` (FEMA-verified real property loss), `accessFunctionalNeeds`, `currentLocation` (e.g. vehicle, shelter, homeless).
- **Used by:** F3 (target: final registrations with home damage per ZIP), F4 (daily counts from `appliedDate`), F8, F5 (registrant mix).
- **Access:** No key. Max 10,000 records per call; page with `$skip`; `$count=true` returns the total. Weekly refresh; covers disasters declared more than 30 days ago (so historical only).
- **Limitations:** Valid registrants only; invalid or withdrawn applications are excluded. ZIP is user-entered and needs cleaning (5 digits). It is a FEMA signal, not State Farm claims; the pitch should say State Farm would swap in its own first-notice-of-loss feed.
- **Alternative:** OpenFEMA Housing Assistance Program Data (Owners and Renters v2) gives registrations aggregated by ZIP per disaster, but without dates, so it can serve only as a check on totals.

```python
import requests, pandas as pd

BASE = "https://www.fema.gov/api/open/v2/"

def openfema(entity, flt, select, page=10_000):
    rows, skip = [], 0
    while True:
        r = requests.get(BASE + entity, params={
            "$filter": flt, "$select": select,
            "$top": page, "$skip": skip, "$format": "json"},
timeout=120)
        r.raise_for_status()
        batch = r.json()[entity]
        rows += batch
        if len(batch) < page:
            return pd.DataFrame(rows)
        skip += page

regs = openfema(
    "IndividualsAndHouseholdsProgramValidRegistrations",
    "disasterNumber eq 4332",   # Harvey; confirm in
DisasterDeclarationsSummaries

"appliedDate,damagedZipCode,censusGeoid,applicantAge,residenceTyp
e,"

"registrationMethod,homeDamage,waterLevel,destroyed,rpfvl,current
Location")
regs.to_parquet("data/raw/ihp_4332.parquet")
```

Harvey produced a very large number of registrations, so this loop makes many calls; run it once, save to Parquet, and never call it during the demo.

6.2 OpenFEMA NFIP Redacted Claims v2

- **Endpoint:** `https://www.fema.gov/api/open/v2/FimaNfipClaims` (also a full Parquet download, about 2.7 million rows).
- **Provides:** `dateOfLoss`, `floodEvent` (event name), `reportedZipCode`, `censusTract`, `countyCode`, `amountPaidOnBuildingClaim`,

`buildingDamageAmount`, `waterDepth`, `occupancyType`, `latitude`/`longitude` rounded to 1 decimal.

- **Used by:** F3 feature (historical claims per housing unit before 2017, a flood-propensity signal), F8 ground truth (Harvey insured damage per ZIP).
- **Limitations:** Only NFIP-insured homes, which are a minority in many Houston ZIPs; there is no claim-filing date, so it is not a timing signal. Coordinates are too coarse for mapping points.

```python
claims = openfema("FimaNfipClaims",
    "state eq 'TX' and dateOfLoss ge '2017-08-24' and dateOfLoss le '2017-09-10'",

"dateOfLoss,floodEvent,reportedZipCode,censusTract,buildingDamageAmount,amountPaidOnBuildingClaim")
```

For the history feature, download the Parquet once and filter with DuckDB: `SELECT reportedZipCode, count(*) FROM 'FimaNfipClaims.parquet' WHERE state='TX' AND yearOfLoss < 2017 GROUP BY 1`.

6.3 FEMA Harvey damage assessments

- **Where:** HydroShare resource "FEMA - Harvey Damage Assessments and Claims," mirroring FEMA's public disaster data site.
- **Provides:** Modeled damaged-building points published daily, from about 78,000 on Aug 28 to about 161,000 on Aug 31, plus auto and property claim points for Aug 25 to Sep 8.
- **Used by:** F2 (damage density per ZIP by day, a real-time-style hazard signal), F8 (independent damage measure).
- **Limitations:** Harvey-specific shapefiles, not an API; modeled, not inspected.
- **Alternative:** FEMA's Harvey 3 m flood depth grid (about 39 GB; too large for a hackathon).

6.4 NOAA/NWS Stage IV precipitation

- **Where:** `https://water.noaa.gov/resources/downloads/precip/stageIV/` and the daily accumulation files under `.../precip/YYYY/MM/DD/` as NetCDF or GeoTIFF for CONUS (archive from mid-2016).
- **Provides:** Radar plus gauge daily rainfall on a ~4 km grid.
- **Used by:** F2 storm-total rainfall per ZIP; F10 live rainfall.
- **Limitations:** 4 km cells are coarse for small ZIPs; rainfall is not flooding (drainage matters), which is why it is combined with high-water marks and damage points. Check the directory listing for the exact file name.

```python
from rasterstats import zonal_stats
import rasterio, numpy as np, geopandas as gpd

days = pd.date_range("2017-08-25", "2017-09-01")
stack = [rasterio.open(f"data/raw/stageiv/{d:%Y%m%d}.tif") for d
in days]
total = sum(s.read(1).astype("float32") for s in stack)   # units
per file metadata
zips =
gpd.read_file("data/raw/zcta_houston.gpkg").to_crs(stack[0].crs)
stats = zonal_stats(zips, total, affine=stack[0].transform,
stats=["mean", "max"], nodata=-9999)
zips["rain_mean"] = [s["mean"] for s in stats]
```

6.5 USGS STN high-water marks

- **Where:** STN Flood Event Data Download portal; choose event "Harvey Aug 2017," data type High-Water Mark, state TX; downloads CSV. HydroShare also hosts the Harvey set (2,123 marks at 1,258 sites).
- **Provides:** Latitude, longitude, height above ground, quality, environment (riverine/coastal).
- **Used by:** F2 (count and max height per ZIP), F7 (exclude flooded candidate sites).
- **Limitations:** Points are surveyed where crews went, so absence of marks is not proof of no flooding.

6.6 CDC/ATSDR Social Vulnerability Index

- **Where:** SVI data download page; choose year 2016 (matches pre-Harvey conditions), Texas, census tracts, CSV.
- **Provides:** Percentile ranks overall (`RPL_THEMES`) and by theme, plus estimates and percentages such as 65+, no vehicle, limited English, mobile homes, and housing units. Confirm exact column names in the documentation PDF for the year you download.
- **Used by:** F3 features, F5 lens, F6 multiplier and action rules.
- **Limitations:** Tract level, ACS 5-year estimates with margins of error.
- **Alternative:** Census ACS API with the same variables.

6.7 Census geography

- **ZCTA polygons:** TIGER/Line or cartographic boundary ZCTA files.
- **ZCTA-to-tract relationship file:** gives each overlap's land area (2020 files list `GEOID_ZCTA5_20`, `GEOID_TRACT_20`, `AREALAND_PART`). SVI 2016 uses 2010

tracts, so use the 2010 ZCTA-to-tract relationship file with the 2010 ZCTA polygons for consistency.
- **Join rule:** tract value → ZIP value weighted by the overlap's share of the tract's land area times tract housing units.

6.8 FEMA National Risk Index

- **Where:** NRI Data Resources page, "All Census Tracts" CSV.
- **Provides:** Riverine and coastal flooding expected annual loss and risk scores by tract.
- **Used by:** F3 feature (baseline flood risk).
- **Limitations:** Current versions reflect recent data, not 2017; acceptable as a structural risk feature.

6.9 Live sources for F10

- **NWS alerts:** `GET https://api.weather.gov/alerts/active?area=TX`, no key, but a descriptive `User-Agent` header is required. Use a generic app name and a team contact address, not a personal one.

```python
r = requests.get("https://api.weather.gov/alerts/active",
params={"area": "TX"},
                 headers={"User-Agent":
"(overlooked-area-detector, team-contact@example.com)",
                          "Accept": "application/geo+json"})
alerts = gpd.GeoDataFrame.from_features(r.json()["features"])  #
some alerts have no polygon
```

- **NHC GIS:** active storm cone, track and warnings as shapefile/KMZ; past storms in the GIS archive.
- **USGS water data:** the legacy WaterServices API is scheduled for decommission on Feb 22, 2027, with degradation starting Nov 2026. Build against `api.waterdata.usgs.gov` (or the updated `dataretrieval` Python package) if you use gauges at all.

6.10 Candidate team sites (F7)

- **Source:** OpenStreetMap through OSMnx: `ox.features_from_place("Harris County, Texas", tags={"amenity": ["parking", "school", "place_of_worship"]})`; keep parking areas over about 5,000 m².
- **Limitations:** Present-day map, not 2017; respect Overpass fair use (download once).
- **Fallback (simulated, labeled):** centroids of the 100 lowest-hazard ZIPs.

6.11 Where real data stops: synthetic and derived data

Need	Approach	Label
State Farm policyholders per ZIP (for the business slide)	Housing units × an assumed market share (e.g. 18%), stated on screen as an assumption	Simulated
Claim intake timing for a live event	Use FEMA registrations as the stand-in; in production, State Farm's own first-notice-of-loss feed	Real stand-in
Training events beyond Harvey	Repeat the pipeline for later Houston-area declarations such as Tropical Storm Imelda (2019) and Hurricane Beryl (2024); confirm disaster numbers in DisasterDeclarationsSummaries	Real historical
Missing ZIP in a registration	Assign from block group via the block-group-to-ZCTA overlap	Derived

7. Database design

Use DuckDB: one embedded file (`oad.duckdb`), no server, reads Parquet directly, and is fast for the group-by and window queries this app runs. It ships inside the backend container and works offline. Geometry is stored as GeoJSON text built by GeoPandas at build time, so no spatial database is needed at run time.

Option	Pros	Cons	Verdict
DuckDB	Zero setup, file-based, reads Parquet, very fast analytics	Single writer; not for many concurrent edits	**Recommended**

PostgreSQL + PostGIS	Real spatial queries, multi-user	Needs hosting, setup time, a free tier that can sleep or expire	Production later
SQLite	Universal	Slower for analytics, no Parquet	Acceptable fallback

Tables

events

Column	Type	Notes
event_id	VARCHAR PK	`harvey-2017`
name	VARCHAR	Hurricane Harvey
disaster_number	INTEGER	FEMA DR number
landfall_date	DATE	Day 0
bbox	VARCHAR	JSON [minx, miny, maxx, maxy]
max_day	INTEGER	30

zips (one row per ZCTA in the study area)

Column	Type	Notes

zip	VARCHAR PK	5 digits
area_name	VARCHAR	Largest city or neighborhood
county_fips	VARCHAR	Majority county
centroid_lat, centroid_lon	DOUBLE	For distances
geom_geojson	VARCHAR	Simplified polygon

zip_static (does not depend on the event)

Column	Type	Notes
zip	VARCHAR PK, FK → zips	
housing_units	INTEGER	From SVI
svi_overall	DOUBLE	0 to 1 percentile
pct_65plus, pct_noveh, pct_limeng, pct_mobile, pct_poverty	DOUBLE	Housing-weighted from tracts
hist_nfip_claims_per_1k_hu	DOUBLE	Claims before the event year
nri_flood_eal_score	DOUBLE	Riverine flood

zip_hazard (per event, per ZIP, per day for time-varying fields)

Column	Type	Notes
event_id	VARCHAR FK → events	PK part
zip	VARCHAR FK → zips	PK part
day	INTEGER	PK part; 0 for storm totals
rain_total_in	DOUBLE	
hwm_count, hwm_max_ft	INTEGER, DOUBLE	
damage_pts_per_1k_hu	DOUBLE	Cumulative to that day
hazard_index	DOUBLE	0 to 1

registrations_daily

Column	Type	Notes
event_id, zip, day	PK	
n_new, n_cum	INTEGER	All valid registrations
n_cum_home_damage	INTEGER	`homeDamage = true`
n_cum_65plus, n_cum_mobile	INTEGER	Registrant mix

n_cum_callcenter	INTEGER	Proxy for people offline

predictions

Column	Type	Notes
event_id, zip, model_version	PK	
expected_final	DOUBLE	Out-of-fold
lo80, hi80	DOUBLE	Interval
contributions_json	VARCHAR	Feature → contribution

gap_scores

Column	Type	Notes
event_id, zip, day	PK	
expected_by_day, observed	DOUBLE, INTEGER	
gap_score, unmet_estimate, p_value	DOUBLE	
significant	BOOLEAN	
priority, rank	DOUBLE, INTEGER	

actions_json, reasons_json	VARCHAR

nfip_claims_event (event, zip PK; claims count, building damage sum) · **sites** (site_id PK, name, kind, lat, lon, zip FK, area_m2, source) · **plans** (plan_id PK, event_id FK, day, k, radius_km, sites_json, coverage_pct, created_at) · **backtest** (event_id, day, method, k PK; precision, recall, spearman) · **model_runs** (model_version PK, algorithm, params_json, cv_deviance, cv_spearman, trained_at).

Index: DuckDB uses zone maps automatically; sort `gap_scores` and `registrations_daily` by `(event_id, day, zip)` when writing so day queries scan little data.

Relationships

- `events` 1 → many `zip_hazard`, `registrations_daily`, `predictions`, `gap_scores`, `nfip_claims_event`, `plans`, `backtest`
- `zips` 1 → 1 `zip_static`; 1 → many of every per-event table; 1 → many `sites`
- `model_runs` 1 → many `predictions`

The diagram below shows these as an entity-relationship map.

[embed: node/02e6a0a4-94fe]

events and zips sit on either side; the five per-event tables in the middle carry both keys, and gap_scores is what the app reads most.

Sample records

JSON


```
{"table": "gap_scores", "event_id": "harvey-2017", "zip":
"77xxx", "day": 5,
 "expected_by_day": 1178.0, "observed": 410, "gap_score": 0.652,
"unmet_estimate": 768.0,
 "p_value": 1.2e-120, "significant": true, "priority": 1083.0,
"rank": 1,
 "actions_json":
"[\"door_to_door_canvass\",\"welfare_check_65plus\"]"}
```


JSON


```
{"table": "zip_static", "zip": "77xxx", "housing_units": 12040,
"svi_overall": 0.82,
```

```
 "pct_65plus": 0.17, "pct_noveh": 0.14, "pct_limeng": 0.21,
"pct_mobile": 0.03,
 "hist_nfip_claims_per_1k_hu": 41.5, "nri_flood_eal_score": 88.2}
```

(Illustrative values.)

How data moves

1. **Pipeline → DB:** scripts write Parquet in `data/processed/`, then `08_export.py` runs `CREATE TABLE ... AS SELECT * FROM 'data/processed/x.parquet'` for each table.
2. **ML → DB:** `train_expected.py` writes `predictions` and `model_runs`; `06_scores.py` reads them with `registrations_daily` to write `gap_scores`.
3. **DB → API:** read-only connection; each endpoint is one parameterized query, returned as Pydantic models or GeoJSON.
4. **API → frontend:** JSON over HTTPS; the frontend never touches the DB.
5. **Frontend → API → DB (only write):** `POST /plan` saves the plan in `plans` (the API opens a short write connection, or keeps plans in memory when the DB is read-only on the host).

8. AI and machine learning

Only one part needs a trained model: predicting expected need per ZIP (F3). Everything else is better done with plain statistics and algorithms, which are faster to build, easier to explain to judges, and harder to break.

Feature	Approach	Why
F3 Expected need	**Machine learning** (Poisson GLM, gradient boosting challenger)	Many interacting inputs; needs a calibrated count prediction
F4 Gap detection	Statistics (ratio + Poisson test)	Transparent; a model here would hide the logic
F6 Actions	Rule table	Responders need predictable, auditable rules

F7 Placement	Greedy optimization	Classic facility-location problem with known guarantees
F9 Brief summary	Optional LLM	Only rewords numbers already computed; template fallback

8.1 Expected-need model (F3)

Problem. For each ZIP z, predict y_z = the final number of FEMA registrations reporting home damage (counted at day 60), using only information available early in the event.

Unit and size. ZCTAs in the Harvey-declared counties around Houston: several hundred rows. Small data, so a simple model is the right default.

Features (all available within about 3 days of landfall, or before the event):

Feature	Source	Transform
log housing units	SVI	Used as exposure offset
rain_total_in	Stage IV	Raw and squared
hwm_max_ft, hwm_count	USGS STN	log1p
damage_pts_per_1k_hu at day 3	FEMA modeled assessments	log1p
hist_nfip_claims_per_1k_hu (pre-2017)	NFIP	log1p
nri_flood_eal_score	NRI	Raw

pct_mobile, pct_noveh, pct_65plus, pct_limeng, pct_poverty	SVI	Raw

No Harvey registration counts go into the features; that would leak the answer.

Target preparation.

```python
regs["applied"] =
pd.to_datetime(regs["appliedDate"]).dt.tz_localize(None)
regs["day"] = (regs["applied"] -
pd.Timestamp("2017-08-25")).dt.days
regs["zip"] =
regs["damagedZipCode"].astype(str).str.extract(r"(\d{5})")[0]
final = (regs[(regs.day <= 60) & regs.homeDamage]
         .groupby("zip").size().rename("y"))
```

Algorithms.

- **Primary: Poisson regression (GLM)** — `sklearn.linear_model.PoissonRegressor(alpha=1e-3)` on standardized features, with log(housing units) included as a feature (or as an offset via `statsmodels` GLM). It models log(expected registrations) = sum of weighted features, which keeps "why" explanations exact and simple.
- **Challenger: gradient-boosted trees** — `HistGradientBoostingRegressor(loss="poisson", max_depth=3, learning_rate=0.05, max_iter=300)`, which can capture thresholds such as "rain above 30 inches."
- Pick the winner by cross-validated Poisson deviance; keep the GLM if the gap is small, because it explains itself.

Training with spatial cross-validation. Neighboring ZIPs share flooding, so random splits overstate accuracy. Cluster ZIP centroids into 6 groups with k-means, then use `GroupKFold(6)`. Each ZIP's prediction comes from a model that never saw its cluster (out-of-fold). Those out-of-fold predictions are what the app uses as `expected_final`.

```python
from sklearn.model_selection import GroupKFold
from sklearn.cluster import KMeans
```

```
groups = KMeans(6, n_init=10,
random_state=0).fit_predict(X[["lat", "lon"]])
oof = np.zeros(len(X))
for tr, te in GroupKFold(6).split(X, y, groups):
    model = make_pipeline(StandardScaler(),
PoissonRegressor(alpha=1e-3, max_iter=1000))
    model.fit(X.iloc[tr][FEATS], y.iloc[tr])
    oof[te] = model.predict(X.iloc[te][FEATS])
```

Uncertainty. An 80% interval from the Poisson distribution around each prediction, widened by the model's overdispersion (ratio of residual variance to mean on out-of-fold data).

Evaluation.

- Poisson deviance (lower is better) vs a baseline of "registrations proportional to housing units."
- Spearman rank correlation between predicted and actual per-housing-unit need (target above 0.6).
- Calibration plot: predicted vs actual by decile.

From prediction to action. expected_final feeds the gap calculation (F4) and the remaining-need forecast (F9). The GLM contribution of each feature (coefficient × standardized value) becomes the "why flagged" reasons in F5.

Production version (say this in the pitch). Train on several past events (Harvey, Imelda 2019, Beryl 2024) and predict a new event before any registrations arrive. The hackathon version demonstrates the method within Harvey using spatial cross-validation.

8.2 Gap detection (F4) — statistics, not ML

For day N: R_N = Σ observed_N ÷ Σ expected_final across all ZIPs; expected_by_day_z = expected_final_z × R_N. Then p_z = PoissonCDF(observed_z; λ = expected_by_day_z). This self-calibrates to the event's overall pace, so a ZIP is flagged only when it is quiet relative to its neighbors' pace, not because day 2 is early for everyone.

8.3 Optional LLM for the brief (F9)

- **Input:** the day's JSON (top 10 ZIPs, actions, sites, staffing numbers).
- **Prompt rules:** use only numbers given; no new facts; 120 words; plain language.
- **Check:** after generation, verify every number in the text appears in the input JSON; if not, fall back to the template.
- **Why optional:** it adds polish to the brief but no decision depends on it.

What not to build

- Image models on satellite photos: impressive, but labeled data prep alone exceeds a hackathon.
- Chatbots for victims: contradict the "no victim participation" constraint.
- Deep learning on a few hundred rows: overfits and cannot be explained.

9. Decision logic and action generation

Raw data becomes an action in five steps: measure the hazard, predict the need, compare need with requests, rank by unmet households weighted by vulnerability, then attach a response type and a location. Each step has one formula a judge can follow.

The chain

1. **Hazard index (F2):** hazard_index = mean of percentile ranks of rainfall, high-water-mark height and damage-point density.
2. **Expected need (F3):** expected_final = model(hazard, housing, history, vulnerability).
3. **Pace-adjusted expectation (F4):** R_N = Σ observed_N ÷ Σ expected_final; expected_by_day = expected_final × R_N.
4. **Gap:** gap_score = max(0, 1 − observed ÷ expected_by_day); unmet = max(0, expected_by_day − observed); significant if PoissonCDF(observed; expected_by_day) < 0.01 and expected_by_day ≥ 20.
5. **Priority (F6):** priority = unmet × (1 + 0.5 × SVI) × (1.0 if significant else 0.5).
6. **Actions:** rule table below.
7. **Placement (F7):** greedy max-coverage over priority weights.
8. **Staffing (F9):** remaining need ÷ visits per adjuster per day.

Priority uses **unmet households**, not the gap percentage alone, so a tiny ZIP that is 80% quiet does not outrank a large ZIP with hundreds of missing households.

Worked example (illustrative numbers, day 5)

Suppose region-wide on day 5, 120,000 registrations have arrived against 400,000 expected in total, so R_5 = 0.30.

Step	ZIP A	ZIP B	ZIP C
Housing units	12,040	9,800	900
expected_final (model)	3,927	3,000	200

expected_by_day = × 0.30	1,178	900	60
observed by day 5	410	820	12
observed ÷ expected	0.35	0.91	0.20
gap_score	0.65	0.09	0.80
unmet	768	80	48
Poisson p	far below 0.001	about 0.004	far below 0.001
significant	yes	yes	yes
SVI	0.82	0.40	0.95
multiplier 1 + 0.5 × SVI	1.41	1.20	1.475
priority	768 × 1.41 = **1,083**	80 × 1.20 = **96**	48 × 1.475 = **71**
rank	1	2	3

Reading: ZIP A is missing roughly 770 households' requests and is highly vulnerable, so it goes first. ZIP C is the quietest in percentage terms but small; it still appears in the list and gets a welfare-check action.

Action rules

Percentiles are relative to Texas tracts from SVI. A ZIP can get several actions; the first matching row is the lead action.

Condition	Action	Why
significant and unmet ≥ 300	**Deploy mobile claims unit / canvass team**	Large missing volume
pct_65plus ≥ 75th percentile	**Welfare checks for older residents**	Less likely to use apps or travel
pct_noveh ≥ 75th percentile	**On-site assistance; transport to center**	Cannot reach a claims center
pct_limeng ≥ 75th percentile	**Multilingual outreach (Spanish, Vietnamese, etc.)**	Language barrier to registering
pct_mobile ≥ 75th percentile	**Priority structural inspections**	Mobile homes fail first
hazard_index ≥ 0.8 and damage points rising	**Coordinate access with local responders**	Area may still be flooded
gap_score < 0.2	**Monitor**	On pace

How the outputs fit together

On day 5 the app produces: the ranked list with lead actions (F6) → a plan placing k teams to cover the top-priority ZIPs from dry sites (F7) → a brief stating "Remaining expected need: 280,000 households; with 6 visits per adjuster per day, 1,000 adjusters clear it in about 47 days" (F9, illustrative) → the Validation page showing how often this ranking was right in Harvey (F8).

Guardrails

- ZIPs with expected_by_day < 20 are never flagged significant (too little data).
- A ZIP flagged on day 3 but on pace by day 6 drops out automatically; the panel shows "resolved" history.
- Every flagged ZIP shows its numbers and reasons, so a human makes the final call.

10. Technology stack

Python for everything data and API; TypeScript React for the screen; all free and open source.

Layer	Technology	Why it fits	How it is used	Simpler alternative
Language (data, API)	Python 3.11+	Team is learning it; best geo and ML libraries	Pipeline, models, API	—
Data wrangling	pandas, pyarrow	Standard; Parquet I/O	All tables	Polars (faster, newer API)
Geospatial	GeoPandas, Shapely	Spatial joins and overlays	ZIP joins, site filtering	—
Raster	rasterio, rasterstats	Read GeoTIFF, zonal stats	Rainfall per ZIP	Use Stage IV point grid instead of rasters
Database	DuckDB	Embedded, fast, reads Parquet	`oad.duckdb`	SQLite
ML	scikit-learn	Poisson GLM, boosting, CV in one library	F3	statsmodels for the GLM only
Statistics	SciPy	Poisson test	F4	—
Optimization	NumPy (own greedy code)	30 lines, no solver needed	F7	PuLP for an exact solve

Road network (stretch)	OSMnx	Drive-time radius	F7 v2	Haversine distance
API	FastAPI + Uvicorn + Pydantic	Typed, fast, auto docs	All endpoints	Flask
PDF	Jinja2 + WeasyPrint	HTML template to PDF	F9 brief	Markdown download only
LLM (optional)	Anthropic Python SDK	Short brief summary	F9	Template sentence
Frontend framework	React 18 + TypeScript + Vite	Fast dev server, typed props	All pages	Streamlit (whole app in Python)
Map	MapLibre GL JS via react-map-gl	Free, GPU, no token	Choropleths, pins	Leaflet + react-leaflet
Basemap tiles	CARTO Positron / Dark Matter or OpenFreeMap	Free, clean	Under the ZIP layer	OpenStreetMap standard tiles
Charts	Recharts	Simple React charts	Curves, bars	Chart.js
UI kit	Tailwind CSS + shadcn/ui	Polished components fast	Cards, slider, tabs	Plain CSS
Data fetching	TanStack Query	Caching per day	All requests	fetch + useEffect

Routing	React Router	URL holds event and day	Pages	—
Auth	None for the demo	Public data only; fewer failure points	—	Add Auth0/Clerk later if needed
Testing	pytest, Vitest, Playwright (one smoke test)	Fast checks	Section 14	pytest only
Tooling	uv or pip + venv, Ruff, Prettier, GitHub	Reproducible env, clean code	Repo	—
Containers	Docker	Same build locally and on host	Backend image	Run Uvicorn directly
Hosting	Vercel (frontend), Render or Fly.io (backend)	Free tiers	Section 15	Laptop demo with ngrok

If the team is small

Two people or fewer: build the MVP as one Streamlit app using the same pipeline and DuckDB file, with `streamlit-folium` for the map. You lose the smooth slider and custom layout but keep all the logic. Everything in sections 6 to 9 still applies.

11. Project structure

One repository with four top-level parts: `pipeline/` builds data, `ml/` trains models, `backend/` serves the API, `frontend/` is the app. Shared scoring code lives in `backend/core/` and is imported by the pipeline so the numbers are computed one way only.

None

```
overlooked-area-detector/
├── README.md                    # setup, how to run, data sources
and labels
├── .env.example                 # OFFLINE, ANTHROPIC_API_KEY,
NWS_USER_AGENT, CORS_ORIGINS
├── Makefile                     # make data | make train | make
scores | make api | make web
├── config/
│   └── events.yaml              # event_id, disaster number,
landfall date, counties, bbox
├── data/                        # gitignored except snapshots
│   ├── raw/                     # downloaded files, never edited
│   ├── processed/               # cleaned Parquet per table
│   ├── oad.duckdb               # built database
│   └── snapshots/               # static JSON per event/day for
offline mode (committed)
├── pipeline/
│   ├── 01_download.py           # OpenFEMA pulls, file downloads,
checksums
│   ├── 02_geography.py          # ZCTAs, tract overlaps, simplify
geometry
│   ├── 03_features.py           # zip_static, zip_hazard
(rasterstats, spatial joins)
│   ├── 04_registrations.py      # registrations_daily from
appliedDate
│   ├── 05_sites.py              # OSM candidate sites
│   ├── 06_scores.py             # gap_scores, priorities, actions
(uses backend/core)
│   ├── 07_backtest.py           # validation metrics
│   ├── 08_export.py             # DuckDB + snapshots
│   └── utils/openfema.py        # paging client
├── ml/
│   ├── features.py              # feature list and transforms
│   ├── train_expected.py        # spatial CV, model choice,
out-of-fold predictions
│   ├── evaluate.py              # deviance, Spearman, calibration
plot
│   ├── models/                  # saved model + model_card.md
│   └── notebooks/eda.ipynb      # exploration only
```

```
├── backend/
│   ├── Dockerfile
│   ├── pyproject.toml
│   ├── api/
│   │   ├── main.py
│   │   └── routers/ events.py maps.py zips.py plan.py
validation.py brief.py live.py
│   ├── core/
│   │   ├── db.py              # DuckDB connection, offline
switch
│   │   ├── scoring.py         # gap, priority, actions
│   │   ├── optimizer.py       # greedy max-coverage
│   │   ├── brief.py           # template, optional LLM, PDF
│   │   └── schemas.py         # Pydantic models
│   ├── templates/brief.html
│   └── tests/                 # pytest: scoring, optimizer, API
contracts
└── frontend/
    ├── package.json
    ├── index.html
    └── src/
        ├── main.tsx, App.tsx  # router
        ├── api/client.ts      # typed fetch functions
        ├── hooks/             # useEvent, useMapDay,
usePriorities
        ├── pages/             # Landing, MapDashboard, Plan,
Validation, Brief
        ├── components/
        │   ├── map/           # ZipMap, Legend, Tooltip,
SitePins
        │   ├── panels/        # PriorityQueue, ZipDetail,
KpiStrip
        │   ├── controls/      # DaySlider, LayerSwitch, PlanForm
        │   └── charts/        # RegistrationCurve, BacktestBars,
ForecastChart
        ├── lib/format.ts      # numbers, dates, data-label
badges
        └── styles/
```

Responsibilities in one line each

- **config/events.yaml:** the only place event facts live; adding Imelda means adding one block.
- **data/raw:** immutable; rerunning the pipeline must not require re-downloading.
- **pipeline/:** numbered scripts run in order; each reads the previous step's Parquet and writes its own.
- **ml/:** training is separate from serving; the app reads predictions, never trains.
- **backend/core:** business logic with no web code, so it is unit-testable and shared.
- **backend/api/routers:** thin HTTP layer only.
- **frontend/src/api:** the single place URLs are defined; components never call fetch directly.
- **data/snapshots:** the demo's safety net (section 15).

12. Implementation roadmap

Eight phases, ordered so that something demoable exists after phase 4 and every later phase only adds value. Hours are rough team-hours for a 4-person team.

Phase 0 · Setup (≈1 h)

- **Build:** repo skeleton (section 11), `events.yaml`, Python env, Vite app, FastAPI "hello" endpoint.
- **Why now:** everyone can work in parallel from a shared structure.
- **Tools:** Git, uv/pip, Node, Vite.
- **Test:** `make api` returns `/health`; `make web` shows a blank map.
- **Done looks like:** a MapLibre map of Houston in the browser.

Phase 1 · Data acquisition (≈4 h) — data lead

- **Build:** `01_download.py` for IHP registrations (DR-4332), NFIP claims (Parquet), SVI 2016 Texas, ZCTA shapes and relationship file, Stage IV days, USGS high-water marks, FEMA damage points, NRI.
- **Why now:** everything depends on data; downloads are the slowest and riskiest step.
- **Test:** row counts logged; registrations have dates from late Aug 2017 onward; every file in `data/raw/` opens.
- **Done looks like:** `data/raw/` complete and backed up to a shared drive.

Phase 2 · Geography and features (≈4 h) — data lead + ML lead

- **Build:** `02_geography.py`, `03_features.py`, `04_registrations.py`.
- **Depends on:** Phase 1.
- **Test:** every study ZIP has housing units, SVI and rainfall; sum of daily registrations equals raw count after ZIP cleaning (report the share dropped).
- **Done looks like:** Parquet tables for `zips`, `zip_static`, `zip_hazard`, `registrations_daily`.

Phase 3 · Model and scores (≈4 h) — ML lead

- **Build:** `train_expected.py` with spatial CV; `06_scores.py` for days 1 to 30.
- **Depends on:** Phase 2.
- **Test:** out-of-fold Spearman reported; sum of expected_by_day equals sum of observed each day (by construction of R_N); hand-check three ZIPs against the formula.
- **Done looks like:** `predictions` and `gap_scores` written; `08_export.py` builds `oad.duckdb`.

Phase 4 · API + map MVP (≈5 h) — backend + frontend leads

- **Build:** `/events`, `/map`, `/kpis`, `/priorities`, `/zips/{zip}`; map with gap layer, slider, KPI strip, priority queue, ZIP panel.
- **Can start before Phase 3 ends:** backend and frontend use a mock DuckDB file with fake scores generated by a 20-line script, then swap in the real file.
- **Test:** pytest contract tests; slider from day 1 to 30 with no errors; numbers match DuckDB queries.
- **Done looks like:** the core demo works end to end. **This is the MVP checkpoint.**

Phase 5 · Placement and validation (≈4 h)

- **Build:** `05_sites.py`, `optimizer.py`, `/plan`, Plan page; `07_backtest.py`, Validation page.
- **Depends on:** Phase 3 (scores).
- **Test:** optimizer unit test on a 5-ZIP toy case with a known answer; backtest numbers reproducible.

Phase 6 · Brief and polish (≈3 h)

- **Build:** forecast chart, brief template and PDF, data badges, dark theme, play animation, loading skeletons.
- **Test:** a fresh browser loads the demo in under 3 seconds; brief opens and numbers match the screen.

Phase 7 · Deploy, snapshots, rehearsal (≈3 h)

- **Build:** Docker image, deploy (section 15), offline snapshots, recorded backup video, pitch rehearsal.
- **Test:** demo runs with Wi-Fi off from snapshots.

Optional Phase 8 · Live watch (≈3 h)

- NWS alerts proxy and Live tab, only if everything else is done.

Who does what, in parallel

Role	Phases 0 to 3	Phases 4 to 7
Data lead	Downloads, geography, features, registrations	Backtest, snapshots
ML lead	Feature design, model, gap scores	Brief summary logic, model card, help backend
Backend lead	FastAPI skeleton, mock DB, schemas	All endpoints, optimizer, deploy
Frontend lead	Map shell, slider, layout with mock data	All pages, polish, play animation
Everyone	—	Pitch (slides, script, rehearsal)

The mock-data trick is what lets frontend and backend work from hour one instead of waiting for the pipeline.

13. MVP, impressive additions and future work

The strongest realistic MVP is: Harvey replay, gap map with day slider, priority queue with actions, and ZIP detail with reasons. Add the Validation page next, because proof that it works beats any extra feature.

Tier 1 · Essential MVP (must work)

Feature	Usefulness	Innovation	Complexity	Why essential
F1 Replay + day slider	High	Medium	Low	Shows change over time

F2 Hazard layer	High	Low	Medium	Separates "quiet because safe" from "quiet because stuck"
F3 Expected need	High	Medium	Medium	The baseline for the gap
F4 Gap detector	Very high	**High**	Medium	Directly answers "no one overlooked"
F5 Why flagged	High	Medium	Low	Makes results trustworthy
F6 Priority + actions	Very high	Medium	Low	Turns insight into a to-do list

Tier 2 · Makes the demo impressive (add in this order)

1. **F8 Validation** — the proof; highest judge impact per hour.
2. **Play animation** of days 0 to 10 — memorable opening.
3. **F7 Team placement** — shows "coordinate assistance."
4. **F9 Brief + staffing forecast** — shows business value to State Farm.

Tier 3 · Future development (mention in the pitch, do not build)

- Ingest State Farm's own first-notice-of-loss and call-center volume in place of FEMA registrations, so it runs live from hour one.
- Train across many past events for true before-registration predictions; add wind and wildfire hazards.
- Road-network drive times and live road closures for placement.
- Block-group resolution where data allows.
- Satellite change detection as an extra hazard feature.
- Integration with partner agencies (Red Cross, county emergency management) through a shared priority feed.

What to cut first if time runs short

In this order: Live watch → PDF (keep Markdown) → LLM summary → road networks → gradient-boosting challenger (keep the GLM) → candidate sites from OSM (use simulated centroids).

14. Testing and validation

Test in three layers: data checks that catch bad joins, unit tests on the scoring and optimizer math, and a backtest that shows the recommendations were right in hindsight.

Data checks (run at the end of each pipeline script)

Check	Pass condition
Registration dates	Earliest `appliedDate` on or after the declaration date; no future dates
ZIP cleaning loss	Under 3% of registrations dropped for bad or out-of-area ZIPs (log the number)
Conservation	Σ daily `n_new` per ZIP = raw count for that ZIP
Housing-unit join	Σ ZIP housing units within ±2% of Σ tract housing units for the counties
Coverage	Every study ZIP has rainfall, SVI and housing units; no NaN in features
Rainfall sanity	Storm totals for Houston ZIPs in the tens of inches, matching published Harvey totals
Geometry	All polygons valid after simplification; GeoJSON under 2 MB

Use `pandera` or plain `assert` statements; fail the pipeline loudly.

Unit tests (pytest)

- **Gap math:** expected 100, observed 30 → gap 0.70, unmet 70, significant; expected 10, observed 0 → not significant (below 20).
- **Pace calibration:** for any day, Σ expected_by_day = Σ observed.
- **Priority:** the section 9 table reproduces exactly.

- **Action rules:** a ZIP above the 65+ threshold gets the welfare-check action; a ZIP with gap < 0.2 gets "Monitor."
- **Optimizer:** five ZIPs on a line with known weights, k = 1 picks the middle site; flooded candidates are never chosen.

API tests

- FastAPI `TestClient` contract tests: each endpoint returns 200 and matches its Pydantic schema; unknown event returns 404; day out of range returns 422.
- Performance: `/map` under 300 ms locally.

Model validation

- Spatial CV Poisson deviance below the housing-units-only baseline.
- Out-of-fold Spearman between predicted and actual need per housing unit above 0.6.
- Calibration plot: predicted vs actual by decile close to the diagonal.
- Residual map: no large cluster of all-positive or all-negative errors (if there is one, add a feature).

Is the recommendation meaningful? (backtest, F8)

1. Define overlooked ZIPs without the model: top 30% by independent damage per housing unit (NFIP building damage + FEMA modeled damage points), and bottom third by share of final registrations received by day 5.
2. Rank ZIPs on day 3, 5 and 7 by: our priority; rainfall only; SVI only; registrations so far (low first).
3. Report precision@10 and @25 for each method.
4. Pass condition: our method beats every single-signal baseline on day 5. If it doesn't, report honestly and show where it helps.

Frontend and end-to-end

- Vitest for formatting helpers and data-badge logic.
- One Playwright smoke test: open landing → open Harvey → move slider to day 5 → click top queue item → panel shows matching ZIP → run plan with k = 3 → three pins appear.
- Manual check on a projector-sized screen and on a laptop at 1366 × 768.

Realistic test scenarios

Scenario	Expected behavior

Day 1 (very few registrations)	Few or no significant flags; KPI says "too early"; hazard layer is the main view
Day 5	Clear cluster of flagged ZIPs; queue stable over repeated loads
Day 20	Most gaps closed; remaining flags highlight persistent under-registration
ZIP with high rain but on pace	Not flagged; panel says "on pace"
Offline mode	Same screens load from snapshots; badge says "Offline snapshot"
Backend down	Frontend shows a clear message and switches to snapshots

15. Deployment and hosting

Host the frontend on Vercel and the backend as a Docker container on Render, with the DuckDB file baked into the image. Then make the demo independent of both: a laptop copy and static snapshots that work with Wi-Fi off. Free-tier terms change often, so check each provider's current limits when you deploy.

Where each part runs

Part	Recommended	Alternative	Limitation to plan for
Frontend	Vercel (free hobby)	Netlify, GitHub Pages	None significant for a static Vite build

Backend API	Render web service from Dockerfile	Hugging Face Spaces (Docker), Railway, Fly.io	Free instances sleep when idle; first request can take up to a minute
Database	DuckDB file inside the backend image	—	Read-only on host; plans kept in memory
ML	No hosting: predictions precomputed	—	Retraining happens on a laptop
LLM (optional)	Anthropic API from the backend	Template only	Needs a key and network

Deployment steps

1. Run the pipeline locally; confirm `data/oad.duckdb` and `data/snapshots/` exist.
2. Backend Dockerfile: `python:3.11-slim`, install requirements, `COPY backend/ data/oad.duckdb`, run `uvicorn api.main:app --host 0.0.0.0 --port $PORT`.
3. Push to GitHub; on Render create a Web Service from the repo, Docker runtime, set environment variables.
4. Frontend: set `VITE_API_URL` to the Render URL; import the repo into Vercel with root `frontend/`, build `npm run build`, output `dist`.
5. Set `CORS_ORIGINS` on the backend to the Vercel domain.
6. Open both URLs 10 minutes before presenting to wake the backend.

If the DuckDB file exceeds GitHub's file-size limit, use Git LFS or download it at container build from a GitHub Release asset.

Environment variables

Variable	Where	Purpose
`DB_PATH`	Backend	Path to `oad.duckdb`

`OFFLINE`	Backend	`1` = serve snapshots
`CORS_ORIGINS`	Backend	Allowed frontend URLs
`ANTHROPIC_API_KEY`	Backend, optional	Brief summary
`NWS_USER_AGENT`	Backend, optional	Required header for NWS alerts
`VITE_API_URL`	Frontend	Backend base URL
`VITE_OFFLINE`	Frontend	`1` = read `/snapshots/*.json` directly

Key management: keys only in host dashboards and a local `.env` that is gitignored; commit `.env.example` with blank values; never put keys in frontend variables (anything with `VITE_` is public). No data source in the MVP needs a key.

Backup plan (layered)

1. **Hosted:** Vercel + Render.
2. **Laptop:** `make api` and `make web` locally; works with no internet except basemap tiles.
3. **Fully offline:** frontend built with `VITE_OFFLINE=1` reads `data/snapshots/{event}/{day}.json`; a tiny vendored basemap (or a plain background with county outlines) replaces online tiles.
4. **Video:** a 2-minute screen recording of the full demo, ready to play.
5. **Screenshots** of each screen in the slide deck.

The snapshots are generated by `08_export.py`: for each day 0 to 30, it writes the map GeoJSON, KPIs, top 50 priorities and a default plan, about 1 to 2 MB per day.

16. Example scenario: Hurricane Harvey, Harris County area

Harvey made landfall in Texas as a Category 4 hurricane on Aug 25, 2017, then stalled over the Houston area; USGS reports that some areas received more than 60 inches of rain and large areas more than 40. This walkthrough runs the whole app on that event. Facts from sources are marked **Real**; model outputs **Predicted**; anything assumed **Simulated**. Specific ZIP results come from your pipeline run, so the numbers shown for ZIPs here are placeholders in the form you will see.

1. Data that enters the system

Input	Source	Label	What it contains
Registrations for DR-4332	OpenFEMA IHP Valid Registrations	Real	Applicant rows with `appliedDate`, ZIP, damage, age band, residence type
Storm-total rainfall Aug 25 to Sep 1	NOAA Stage IV	Real	~4 km daily grids summed
High-water marks	USGS STN	Real	2,123 marks at 1,258 sites in Texas
Modeled damaged buildings	FEMA damage assessments	Real	Daily layers, e.g. about 78,000 points on Aug 28 rising to about 161,000 on Aug 31
Flood claims	NFIP Redacted Claims	Real	Harvey-period claims by ZIP; pre-2017 history
Vulnerability	CDC SVI 2016	Real	Tract percentages and housing units
Candidate sites	OpenStreetMap	Real (present-day map)	Parking, schools, places of worship

State Farm policyholders per ZIP	Housing units × assumed share	Simulated	Only for the business-value slide

2. Processing

1. ZIPs: ZCTAs intersecting the declared counties around Houston; tract SVI aggregated by housing-unit weights.
2. Hazard: rainfall zonal mean, high-water-mark count and max, damage points per 1,000 housing units on Aug 28 (day 3) → hazard_index.
3. Registrations: cleaned ZIPs; day = appliedDate − Aug 25; cumulative counts per ZIP per day; final target = home-damage registrations by day 60.
4. Model: Poisson GLM, 6-cluster spatial CV → out-of-fold expected_final (**Predicted**).
5. Scores for each day: R_N, expected_by_day, gap, Poisson p, priority, actions (**Predicted**).
6. Sites: drop candidates in high-hazard ZIPs or near high-water marks above 2 ft.
7. Backtest: overlooked label from independent damage and early registration share; precision@k vs baselines.

3. What the user sees, screen by screen

Landing. Card "Hurricane Harvey · Aug 25 to Sep 2017 · DR-4332." Click Open.

Map, day 1 (Aug 26). Hazard layer on by default because few people have registered yet. Deep blue across much of Harris County. KPI tile: "Too early for gap signal."

Map, day 3 (Aug 28). Switch to Gap. A first set of ZIPs turns red. Hover one: "Expected so far ~X · Observed Y · Z% quiet · significant." The queue's top card shows actions such as "Canvass team · Welfare checks 65+."

Press Play, days 3 to 10. Some red ZIPs fade as registrations catch up (on pace). Others stay red for days; these persistent gaps are the story.

Day 5, click the #1 ZIP. Side panel:

- Registration curve: observed line well below the expected curve, marker at day 5.
- Why flagged: e.g. "Rain ~40+ in (Real) · 3 high-water marks above 4 ft (Real) · 9 modeled damaged buildings per 100 homes (Real) · 21% limited English (Real) · Expected need ~X households (Predicted)."
- Actions: "Deploy canvass team," "Spanish-language outreach."

Plan page, day 5, k = 4, radius 10 km. Four pins at dry candidate sites; table shows households in priority ZIPs within reach; coverage gauge e.g. "64% of priority need covered."

Validation page. Precision@10 on day 5 for our method vs rainfall only, SVI only and raw registrations, plus a map of hits and misses (**Real** outcome vs **Predicted** ranking).

Brief. One page: top 10 ZIPs with actions, 4 sites, remaining expected need, staffing at 6 visits per adjuster per day. Download PDF.

4. Recommended actions on day 5 (format)

Rank	ZIP	Unmet (Pred.)	Lead action	Site
1	from run	from run	Mobile claims unit / canvass	Site A
2	from run	from run	Welfare checks, 65+	Site A
3	from run	from run	Spanish and Vietnamese outreach	Site B
…				

5. Value to State Farm

- **Speed:** a ranked list of where to send catastrophe teams on day 3 to 5, before quiet areas would otherwise show up in claim volume.
- **Fairness and goodwill:** reaching vulnerable households that do not call in protects customers and the brand and supports regulators' expectations of fair claims handling.
- **Cost:** earlier contact can shorten displacement and reduce additional-living-expense payouts; staff go where unmet need is largest.
- **Business case (Simulated):** if State Farm insures roughly 18% of homes in the top 25 flagged ZIPs, the slide multiplies unmet households by that share to show how many policyholders the team would reach earlier. State the share as an assumption.
- **Production path:** swap FEMA registrations for State Farm's own first-notice-of-loss feed, and the same pipeline runs live from hour one of the next storm.

17. Final architecture and development checklist

The architecture diagram is in section 1 and the database diagram in section 7; the end-to-end data flow is drawn below. The tables and checklist here are the one-page reference to build from.

End-to-end data flow

[embed: node/3424a3e3-b728]

Registrations skip the model on purpose: they are what the model's expectation is compared against, never an input to it.

Feature-to-technology map

Feature	Pipeline	Model / algorithm	API	Frontend
F1 Replay	events.yaml	—	`/events`, `/kpis`	DaySlider, KpiStrip
F2 Hazard	rasterio, rasterstats, GeoPandas	Percentile index	`/map`	ZipMap layer
F3 Expected need	pandas features	Poisson GLM, GroupKFold	`/zips/{zip}`	ZipDetail
F4 Gap	DuckDB, SciPy	Pace ratio, Poisson test	`/map`, `/priorities`	ZipMap, PriorityQueue
F5 Why flagged	SVI aggregation	GLM contributions	`/zips/{zip}`	ZipDetail bars, reasons
F6 Priority + actions	`core/scoring.py`	Weighted score, rule table	`/priorities`	PriorityQueue
F7 Placement	OSMnx sites	Greedy max-coverage, haversine	`POST /plan`	Plan page, SitePins

F8 Validation	`07_backtest.py`	Precision@k, Spearman	`/backtest`	Validation page
F9 Forecast + brief	—	Arithmetic; optional Claude	`/forecast`, `/brief`	ForecastChart, Brief page
F10 Live watch	—	Polygon intersection	`/live/alerts`	Live tab

Feature-to-data-source map

Feature	Data sources
F1	FEMA disaster declarations (event facts)
F2	NOAA Stage IV, USGS high-water marks, FEMA damage assessments, Census ZCTAs
F3	All F2 sources + CDC SVI + NFIP history + NRI (features); IHP registrations (target)
F4	IHP registrations (`appliedDate`), F3 predictions
F5	CDC SVI, Census relationship files
F6	F4 + F5 outputs
F7	OpenStreetMap sites, USGS high-water marks, F6 priorities

F8	NFIP claims (Harvey), FEMA damage assessments, IHP registrations
F9	F3, F4, F7 outputs
F10	NWS alerts, NHC GIS, Stage IV hourly

Required APIs and datasets

1. OpenFEMA IHP Valid Registrations v2 — core
2. OpenFEMA NFIP Redacted Claims v2 — core
3. OpenFEMA API documentation and developer resources
4. FEMA Harvey Damage Assessments and Claims (HydroShare) — core
5. NOAA/NWS precipitation data access (Stage IV) — core
6. USGS STN Flood Event Data portal and Harvey high-water marks on HydroShare — core
7. CDC/ATSDR SVI data download — core
8. Census relationship files and ZCTA record layouts — core
9. FEMA National Risk Index downloads — core
10. NHC GIS data — optional
11. NWS API documentation — optional
12. USGS Water Data APIs and WaterServices decommission notice — optional
13. Census ACS 5-year API — optional

Prioritized checklist

- Repo skeleton, envs, `events.yaml` (Phase 0)
- Download IHP registrations for DR-4332 and save Parquet
- Download SVI 2016 Texas tracts, ZCTA shapes, relationship file
- Download Stage IV days Aug 25 to Sep 1, 2017; USGS high-water marks; FEMA damage points
- Download NFIP claims Parquet; NRI tract CSV
- Build `zips`, `zip_static`, `zip_hazard`, `registrations_daily`
- Mock DuckDB for frontend/backend to start
- Train Poisson GLM with spatial CV; write out-of-fold predictions
- Compute gap scores, priorities, actions for days 1 to 30
- API: events, map, kpis, priorities, zip detail
- Frontend: map + slider + KPI + queue + ZIP panel (**MVP checkpoint**)
- Backtest and Validation page
- Sites, optimizer, Plan page
- Forecast, brief, polish, play animation
- Deploy; snapshots; offline build; backup video
- Pitch rehearsal with the Harvey storyline

Exact first steps

1. Create the GitHub repo with the folder structure from section 11 and add teammates.
2. In Python, run the `openfema()` paging function from section 6.1 with `disasterNumber eq 4332` on a small `$top` first to see real rows, then the full pull, saved to `data/raw/ihp_4332.parquet`.
3. Confirm the Harvey disaster number and declared counties with a query to `DisasterDeclarationsSummaries` filtered on `disasterNumber eq 4332`.
4. Download SVI 2016 Texas tracts and the 2010 ZCTA polygons; plot Harris County ZIPs colored by SVI in a notebook to confirm the join.
5. Count registrations per ZIP by day and plot five ZIPs' cumulative curves; this is the first look at the gap signal.
6. Meanwhile, the frontend lead scaffolds Vite + MapLibre and renders ZIP polygons from a static GeoJSON file.

Sources

All links above, checked Nov 7, 2026. Harvey facts: USGS Harvey flood inundation data release, NHC Harvey tropical cyclone report.